OBJECT ORIENTED PROGRAMMING – SET 1

Section 1: Fundamentals & Principles

1. Which principle describes hiding internal details?

   a) Inheritance

   b) Abstraction

   c) Encapsulation

   d) Polymorphism

   Answer: b) Abstraction

   Explanation: Abstraction exposes only essential details to the outside

2. Which of the following demonstrates encapsulation in Java?

```
class Account {
 private double balance;
    public void deposit(double amount) {
    if (amount > 0) {
       balance += amount;
    }
 }
 public double getBalance() {
    return balance;
 }
}
```

   a) using private fields and public methods

   b) declaring methods as static

   c) using inheritance

   d) overriding the deposit method

Answer: a)

Explanation: Encapsulation is implemented by keeping class variables private and providing public methods to access and modify them.

3. Which keyword supports compile-time polymorphism?

a) extends

b) implements

c) final

d) static

Answer: c) final

Explanation: final prevents overriding but overloading supports compile-time polymorphism

4. You are designing a login system. Which concept will you use to restrict direct access to user passwords, allowing only specific methods to modify or view them?

a) Inheritance

b) Abstraction

c) Encapsulation

d) Polymorphism

Answer: c)

Explanation: Encapsulation restricts direct access to data by keeping it private and providing public getter/setter methods to control access. This ensures data security and proper validation.

🔹 Section 2: Class & Object Essentials

5. What is the Output for the below code snippet ?

```
class Demo {
   static int count = 0;
   Demo() {
      count++;
```

```
    }

    public static void main(String[] args) {
        Demo d1 = new Demo();
        Demo d2 = new Demo();
        System.out.println(Demo.count);
    }
}
```

a) 0
b) 1
c) 2
d) compilation error

Answer: c)
Explanation: The static variable count is shared across all instances. It gets incremented twice, once for each object created.

6. What does super() do in a constructor?

   a) Calls same class constructor

   b) Calls parent class constructor

   c) Initializes static variables

   d) Creates new object

   Answer: b) Calls parent class constructor

   Explanation: super() invokes superclass constructor

7. Which is true about object creation?

```
class Car {

    String model;

}
```

a) Car model = new Car(); is valid

b) Car = new model(); is valid

c) new Car = model(); is valid

d) object cannot be created without constructor

Answer: a)

Explanation: This is the correct syntax for creating a new Car object and assigning it to a variable named model.

8. Using == with Java strings compares:
   a) Content
   b) Memory address
   c) Length
   d) Hashcode
   Answer: b) Memory address
   Explanation: == checks reference equality, not string contents
9. Which method compares the content of two objects in Java?
   a) compareTo()
   b) equals()
   c) ==
   d) equalsIgnoreCase()
   Answer: b) equals()
   Explanation: equals() checks internal content equality
10. When should hashCode() be overridden in Java?
    a) Only if equals() is overridden
    b) Only for serialization
    c) Always
    d) Never
    Answer: a) Only if equals() is overridden
    Explanation: Equal objects must have same hash code
11. What is the result of the following code?

```
class A {
   int a = 5;
}
public class Main {
```

```
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = new A();
        obj2.a = 10;
        System.out.println(obj1.a);
    }
}
```

a) 10

b) 5

c) 0

d)Compilation error

Answer: b)

Explanation: obj1 and obj2 are different objects. Changing obj2.a does not affect obj1.a.

12. Predict the output :

```
class Sample {
    void display() {
        System.out.println("Hello World");
    }
}
public class Demo {
    public static void main(String[] args) {
        Sample s = null;
```

```
        s.display();
    }
}
```

a) Hello World

b) NullPointerException

c) Compilation error

d) Runtime error

Answer: b)

Explanation: Calling an instance method on a null reference throws `NullPointerException`.

🔹 Section 3: Inheritance & Interfaces

13. What is the output?

```
class SuperClass {
    static void display() {
        System.out.println("SuperClass display");
    }
}

class SubClass extends SuperClass {
    static void display() {
        System.out.println("SubClass display");
    }
}

public class Main {
    public static void main(String[] args) {
        SuperClass obj = new SubClass();
        obj.display();
```

```
    }
}
```

a) SuperClass display

b) SubClass display

c) Compilation error

d) Runtime error

Answer: a)

Explanation: Static methods are not polymorphic. Method resolution is based on the reference type, not the object type.

14. An inner class defined without a name is called:

a) Nested class

b) Local class

c) Anonymous inner class

d) Public class

Answer: c) Anonymous inner class

Explanation: Anonymous inner class is unnamed

15. What happens if two interfaces define the same method signature?

```
interface A {
    void test();
}
interface B {
    void test();
}
class Impl implements A, B {
    public void test() {
        System.out.println("Implementing test");
    }
}
```

a) Compilation error due to conflict

b) Only A's method is inherited

c) Successfully compiles and runs

d) Runtime ambiguity error

Answer: c)

Explanation: Since both methods are identical in signature and abstract, the compiler allows it. *Impl* provides a single implementation.

16. Choose the correct behavior:

```
interface A {
    default void show() {
        System.out.println("A");
    }
}
interface B {
    default void show() {
        System.out.println("B");
    }
}
class C implements A, B {
    public void show() {
        A.super.show();
    }
}
public class Main {
    public static void main(String[] args) {
        new C().show();
    }
}
```

a) A

b) B

c) Compilation error

d) Runtime error

Answer: a)

Explanation: When two interfaces have conflicting default methods, the implementing class must override the method and resolve ambiguity explicitly.

17. Why does Java not support multiple inheritance through classes?

a) It increases compilation time

b) JVM does not support multiple method calls

c) To avoid ambiguity such as the Diamond Problem

d) Java is a purely procedural language

Answer: c)

Explanation: Multiple class inheritance can cause ambiguity when the same method exists in multiple parent classes (known as the Diamond Problem). Java avoids this by supporting multiple inheritance only through interfaces.

18. Which access modifier will allow a method to be inherited outside the package?

a) private

b) protected

c) default

d) static

Answer: b)

Explanation:

Only protected (or public) allows access to inherited methods across packages. default is package-private.

🔹 Section 4: Polymorphism Details

19. Which of these statements about method overriding is true?

a) Overriding methods must have different parameter types

b) Overriding methods can throw broader exceptions than base class

c) Overriding methods must have same name, return type, and parameter list

d) Overriding methods must be static

Answer: c)

Explanation: Method overriding requires **exact match** in method signature and return type. Return type can be covariant, but not broader.

20. Predict the solution:

```
class Alpha {
   void show() {
      System.out.println("Alpha");
   }
}

class Beta extends Alpha {
   void show(String s) {
      System.out.println("Beta " + s);
   }
}

public class Main {
   public static void main(String[] args) {
      Beta b = new Beta();
      b.show("X");
   }
}
```

a) Alpha

b) Beta X

c) Compilation error

d) Alpha Beta

Answer: b)

Explanation: show(String s) is **overloaded**, not overridden. Since this is a direct call to Beta, it invokes the overloaded method.

21. Keyword instanceof helps to:

    a) Create new instance

    b) Compare objects

    c) Check object type

    d) Cast object

    Answer: c) Check object type

    Explanation: Verifies if object is instance of a class/interface

22. For polymorphism, subclass method must share signature with:

    a) Constructor

    b) Superclass method

    c) Private method

    d) Static method

    Answer: b) Superclass method

    Explanation: Overriding requires same name and parameters

23. What will this code print?

```
class A {
   int num = 10;
   void print() {
      System.out.println("A: " + num);
   }
}

class B extends A {
   int num = 20;
   void print() {
      System.out.println("B: " + num);
   }
}

public class Main {
   public static void main(String[] args) {
      A obj = new B();
      obj.print();
```

```
        System.out.println(obj.num);
    }
}
```

a) B: 20
10

b) A: 10
20

c) B: 10
10

d) B: 20
20

Answer: a)

Explanation: print() is overridden and resolved at runtime. But obj.num is a **field**, not overridden, so it accesses field from A.

24. What will happen

```
class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}
class Child extends Parent {
    Child(int x) {
        System.out.println("Child constructor: " + x);
    }
}
public class Main {
    public static void main(String[] args) {
        new Child(10);
    }
}
```

a) Compilation error (no super constructor)

b) Only Child constructor

c) Parent constructor

Child constructor: 10

d) Runtime error

Answer: c)

Explanation: Java automatically inserts super() in a constructor if not explicitly specified. So, Parent constructor runs before Child.

25. Which is NOT an example of polymorphism?

a) Method overloading

b) Method overriding

c) Constructor chaining

d) Interface-based reference calling implementation

Answer: c)

Explanation: Constructor chaining is related to object creation flow, not polymorphism. The other three are valid polymorphism examples.

Section 5: Constructors & this

26. Which is not a constructor?

a) Default constructor

b) Copy constructor

c) Static constructor

d) Parameterized constructor

Answer: c) Static constructor

Explanation: Java has no static constructors .

27. What does this print ?

```
class Demo {
   Demo() {
      this(5);
      System.out.println("Default constructor");
```

```
    }
    Demo(int x) {
        System.out.println("Param constructor: " + x);
    }
}
public class Main {
    public static void main(String[] args) {
        new Demo();
    }
}
```

a) Default constructor
b) Param constructor: 5
c) Param constructor: 5
Default constructor
d) Compilation error

Answer: c)
Explanation: this(5) calls the parameterized constructor first. After that, the default constructor continues execution.

28. What happens if you call a constructor inside another constructor without using this()?

a) Compiles fine
b) Leads to recursion
c) Compilation error
d) Runtime error

Answer: c)
Explanation: In Java, constructor chaining **must use this()** and it must be the **first statement** in the constructor. Otherwise, compilation fails.

29. Calling one constructor from another uses keyword:

a) this()

b) super()

c) base()

d) parent()

Answer: a) this()

Explanation: this() calls another constructor in same class .

30. In Java, what is a **key rule** when chaining constructors?

a) this() can appear anywhere in the constructor

b) super() must always be called manually

c) this() must be the first line in the constructor

d) You cannot have more than one constructor in a class

Answer: c)

Explanation: When chaining constructors using this(), Java **requires** it to be the **first statement**, otherwise it will result in a compilation error.

31. Constructor overloading allows:

a) Two constructors

b) Multiple constructors with different params

c) No constructor

d) Only private constructors

Answer: b) Multiple constructors with different params

Explanation: Enables different initialization options

🔹 Section 6: Abstraction & abstract

32. Abstract class may contain:

a) Only abstract methods

b) Both abstract and concrete methods

c) No methods

d) Only fields

Answer: b) Both abstract and concrete methods

Explanation: Abstract classes can include regular methods

33. Instantiating an abstract class causes:

a) Compile-time error

b) Runtime error

c) Warning

d) Implicit instantiation

Answer: a) Compile-time error

Explanation: Abstract classes cannot be directly instantiated

34. What happens if a subclass does not implement all abstract methods

```
abstract class Animal {

   abstract void sound();

}

class Dog extends Animal {

   // No implementation of sound()

}
```

a) Runs fine

b) Throws runtime exception

c) Compilation error

d) Automatically implements sound() as empty

Answer: c)

Explanation: If a subclass doesn't implement **all abstract methods**, it must be declared abstract itself. Otherwise, **compilation fails**.

35. What line causes error here?

```
abstract class A {

   abstract void test();
```

```
    void hello() {
        System.out.println("Hi");
    }
}
public class Demo {
    public static void main(String[] args) {
        A a = new A();  // Line X
    }
}
```

a) hello() method
b) test() method
c) Line X
d) No error

Answer: c)
Explanation: You **cannot instantiate** an abstract class directly. Line X tries to do so, resulting in a **compilation error**.

36. Interface methods are by default:
a) Public abstract
b) Private
c) Protected
d) Static
Answer: a) Public abstract
Explanation: Interface methods are implicitly public and abstract.

37. What is the error?

```
abstract class MyClass {
    abstract void doSomething();
    private abstract void hidden();  // Error line
```

```
}
```

a) No error

b) Only public abstract methods allowed

c) Abstract method cannot be private

d) Abstract class cannot be public

Answer: c)

Explanation: Abstract methods must be **accessible to subclasses**, so **private abstract** methods are **not allowed**.

🔹 Section 7: final Keyword

38. Declaring a class as final means:

a) Methods can't be overridden

b) Class can't be subclassed

c) Methods can't be overloaded

d) Variables can't change

Answer: b) Class can't be subclassed

Explanation: Final classes cannot be extended

39. What happens when this code runs?

```
class Parent {
    final void show() {
        System.out.println("Final method");
    }
}
class Child extends Parent {
    void show() {
        System.out.println("Overridden");
```

```
    }
}
```

a) Prints “Overridden”

b) Compilation error

c) Prints “Final method”

d) Runtime error

Answer: b)

Explanation: You cannot override a final method in a subclass. This will cause a compile-time error

40. What is the effect of declaring a parameter final?

```
void display(final int a) {
   a = 20;
}
```

a) Compilation error

b) Runs normally

c) a becomes static

d) a becomes class-level variable

Answer: a)

Explanation: You **cannot reassign** a final parameter within the method. Trying to do so leads to **compilation error**.

41. final on reference means:

a) Object can't change

b) Reference can't change

c) Object is immutable

d) Variable is protected

Answer: b) Reference can't change

Explanation: The reference can't point to another object, though object state can change

42. Applying final on local variable:

a) Prevents change

b) Makes it thread-safe

c) Converts to static

d) Generates compiler warning

Answer: a) Prevents change

Explanation: Local final variables act as constants.

43. What does this code demonstrate?

```
class Example {
   final int x;
   Example() {
      x = 10;
   }
}
```

a) Static block assignment

b) Blank final variable

c) Final method

d) Final class

Answer: b)

Explanation:

A blank final variable is a final variable not initialized at declaration, but must be initialized in the **constructor**.

🔹 Section 8: Exception Handling Basics

44. Which exception is thrown by dividing by zero in Java?

a) IOException

b) ArithmeticException

c) RuntimeException

d) NullPointerException

Answer: b) ArithmeticException

Explanation: Division by zero at runtime triggers ArithmeticException.

45. Checked exceptions must be:

a) Caught or declared

b) Ignored

c) Converted

d) Extending RuntimeException

Answer: a) Caught or declared

Explanation: Compile-time enforcement for checked exceptions.

46. What happens when no catch block matches the exception?

```
public class Test {
    public static void main(String[] args) {
        try {
            int[] a = new int[3];
            a[5] = 10;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception");
        }
    }
}
```

a) Compilation error

b) ArrayIndexOutOfBoundsException

c) Arithmetic Exception

d) Code runs silently

Answer: b)

Explanation:

An ArrayIndexOutOfBoundsException is thrown but **not caught** by the catch block (which only catches ArithmeticException). So, it's unhandled and thrown.

47. Which keyword throws a custom exception?

a) try

b) catch

c) throw

d) finally

Answer: c) throw

Explanation: throw explicitly invokes an exception.

48. Uncaught runtime exceptions:

a) Must be handled

b) Cause compiler error

c) May terminate program

d) Are ignored

Answer: c) May terminate program

Explanation: If not caught, runtime exceptions end execution.

49. What's wrong with this code?

```
public class Test {
  public static void main(String[] args) {
    try {
      int a = 10 / 0;
    }
    System.out.println("End");
  }
}
```

a) Nothing

b) Output: End

c) Compilation error
d) Runtime exception

Answer: c)

Explanation:

try block **must be followed** by either catch or finally. In this code, it's not, so it causes a **compilation error**.

50. What happens if an exception is not caught and not declared?

```
public class Test {
   public static void main(String[] args) {
      FileReader f = new FileReader("test.txt");
   }
}
```

a) Compiles fine
b) Runtime error
c) Compilation error
d) File not found message

Answer: c)

Explanation: FileReader may throw FileNotFoundException, which is **checked**. Since it's not caught or declared, this results in a **compilation error**.